The Power of Ordering -- Search, Insert, Delete

Lecture 11

Data Structures and Algorithms

Part 1: Why Binary Search Trees?

The Search Problem -- A Familiar Dilemma

We have seen two approaches to searching in sorted data:

Approach	Search	Insert	Delete
Unsorted Array	O(n)	O(1)	O(n)
Sorted Array	O(log n) via binary search	**O(n)** -- must shift elements	**O(n)** -- must shift elements
Linked List	O(n)	O(1) at head	O(n) to find

The frustration: Sorted arrays give us fast search (O(log n)), but inserting or deleting an element is expensive because we have to shift everything.

Can we get **O(log n) search AND O(log n) insertion** in the same data structure?

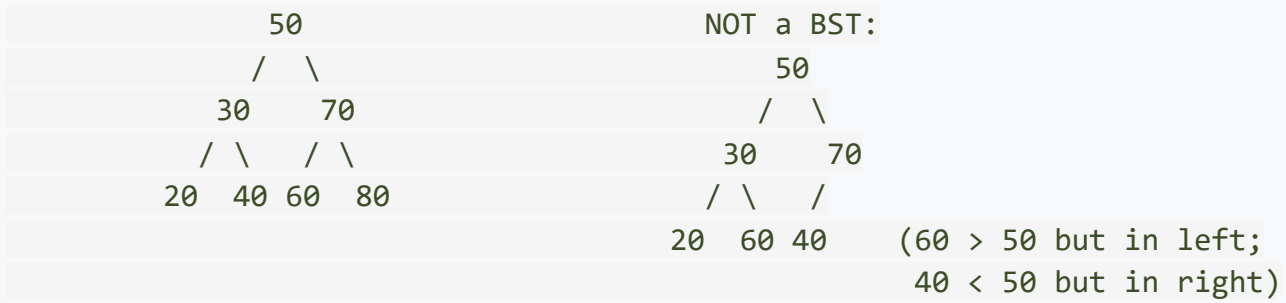
Yes. That is exactly what a Binary Search Tree delivers.

What is a Binary Search Tree?

A **Binary Search Tree (BST)** is a binary tree where every node satisfies a simple ordering rule:

For every node N:

- All values in the **left subtree** of N are **less than** N
- All values in the **right subtree** of N are **greater than** N



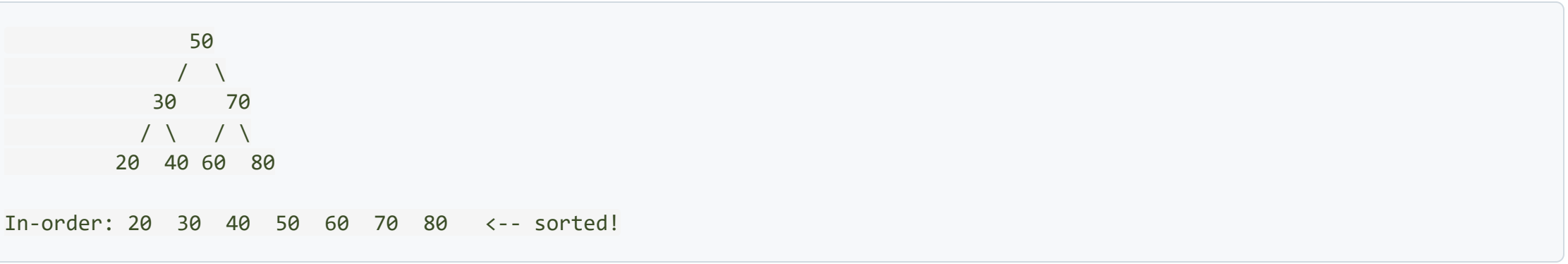
Important: The rule applies to the **entire subtree**, not just the immediate children. Every descendant on the left must be smaller; every descendant on the right must be larger.

The Connection to In-Order Traversal

Recall from the last lecture: in-order traversal visits nodes in order Left, Node, Right.

For a BST, this means we visit all smaller values first (left subtree), then the node itself, then all larger values (right subtree).

Result: In-order traversal of a BST always produces a sorted sequence.



This is not a coincidence -- it is a direct consequence of the BST property. This fact is extremely useful: if you ever need to verify whether a tree is a valid BST, just perform an in-order traversal and check if the output is in ascending order.

Part 2: BST Search

Searching in a BST

The algorithm is beautifully simple: Start at the root. At each node, compare the target value with the node's data. If the target is smaller, go left. If larger, go right. If equal, found it!

```
Search for 40 in:
               50
              /  \
            30    70
           / \   / \
         20  40 60  80

Step 1: 40 < 50  --> go LEFT to 30
Step 2: 40 > 30  --> go RIGHT to 40
Step 3: 40 == 40 --> FOUND!

Only 3 comparisons for 7 nodes!
```

Analogy: This is exactly how binary search works on a sorted array -- at each step, we eliminate half the remaining candidates. But unlike a sorted array, we can also insert and delete efficiently.

BST Search: Code

Recursive Version

```
struct Node* search(struct Node* root, int key) {
    if (root == NULL) return NULL;       // Not found
    if (key == root->data) return root;  // Found!
    if (key < root->data)
        return search(root->left, key);  // Go left
    else
        return search(root->right, key); // Go right
}
```

Iterative Version

```
struct Node* searchIterative(struct Node* root, int key) {
    while (root != NULL) {
        if (key == root->data) return root;
        if (key < root->data) root = root->left;
        else                  root = root->right;
    }
    return NULL;  // Not found
}
```

Both are **O(h)** where h = height of the tree.

Search: What Determines Performance?

The number of comparisons equals the **depth of the node** we are searching for. In the worst case, we travel from the root to a leaf -- that is **O(h)** comparisons.

Best case (balanced tree):

```
          50
         /  \              Height = log2(7) = 2
       30    70            Max comparisons = 3
      / \   / \
    20  40 60  80          O(log n)
```

Worst case (skewed tree):

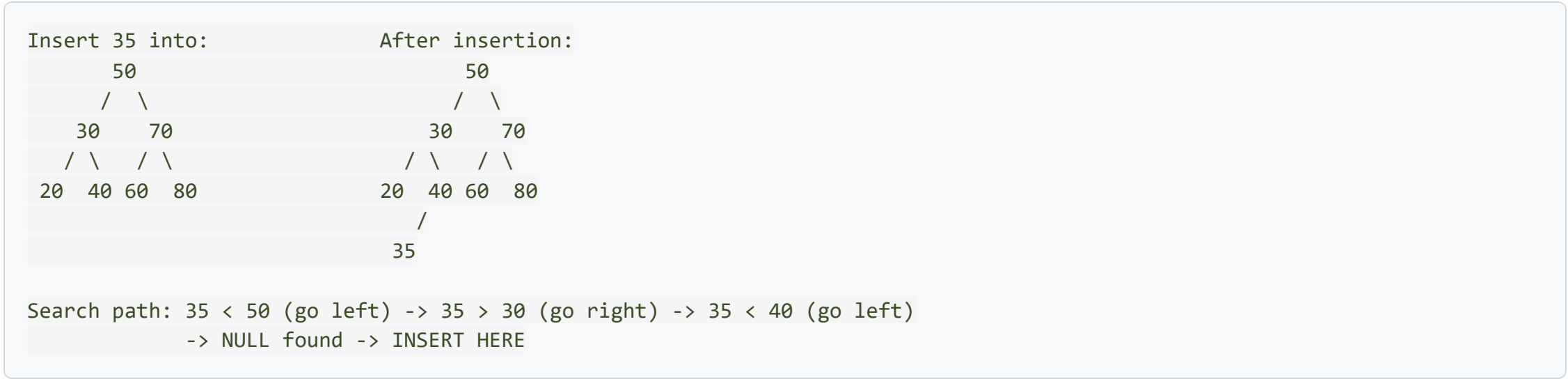
```
    10
     \
      20
       \                  Height = n - 1 = 6
        30                Max comparisons = 7
         \
          40              O(n) -- just a linked list!
           \
            50
             \
              60
               \
                70
```

The shape of the tree determines everything. We will address this problem later with balanced BSTs.

Part 3: BST Insertion

Inserting a New Value

Key insight: A new node is always inserted as a **leaf**. We search for where the node *would* be, and when we reach a NULL pointer, that is where we create the new node.



The new node 35 becomes the left child of 40.

Insertion: Code

```
struct Node* insert(struct Node* root, int value) {
    // Base case: found the insertion spot
    if (root == NULL) {
        return createNode(value);
    }

    // Recursive case: go left or right
    if (value < root->data) {
        root->left = insert(root->left, value);
    } else if (value > root->data) {
        root->right = insert(root->right, value);
    }
    // If value == root->data, it is a duplicate -- do nothing

    return root;
}
```

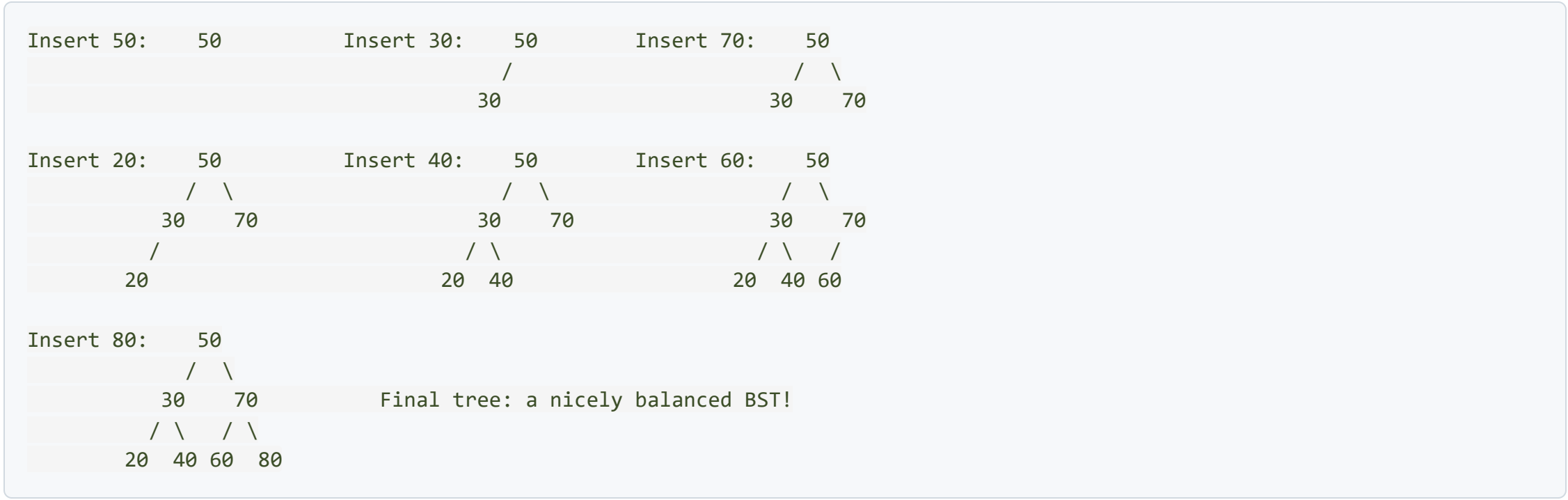
Why `root->left = insert(root->left, value)`?

This pattern lets the recursive call **attach** the new node by returning it. When `root->left` is NULL, `insert(NULL, value)` creates and returns a new node, which then gets assigned to `root->left`.

Complexity: O(h) -- same as search.

Building a BST: Step by Step

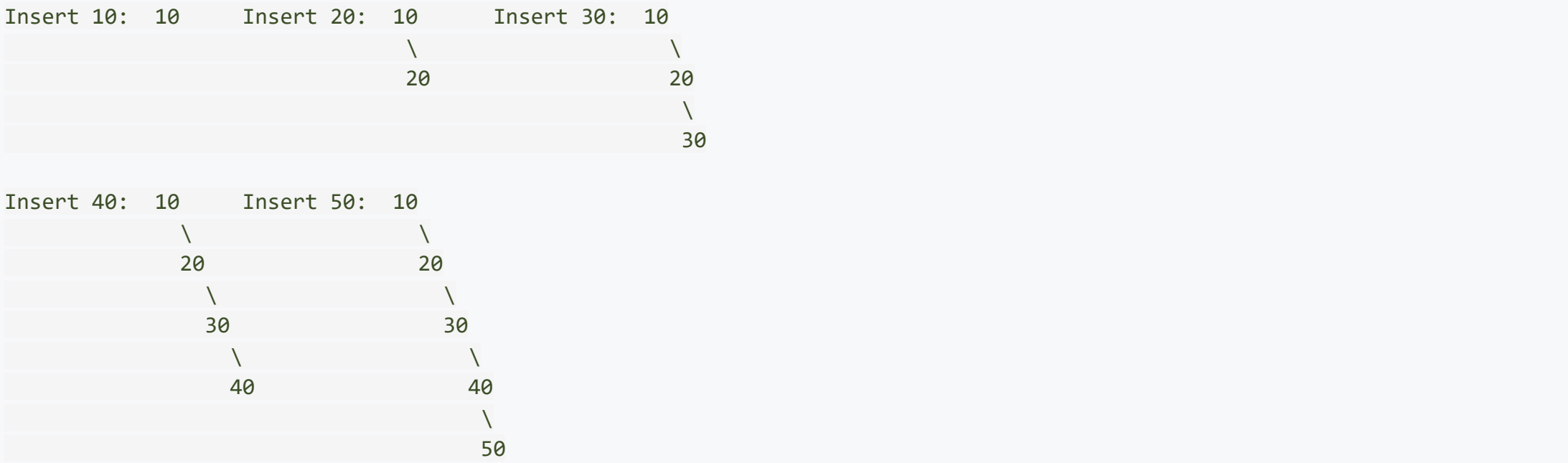
Insert the sequence **50, 30, 70, 20, 40, 60, 80** one at a time:



The order of insertion determines the shape. This same set of values inserted in a different order would produce a different tree.

The Danger of Sorted Input

What happens if we insert **already sorted data**: 10, 20, 30, 40, 50?



The tree degenerates into a **right-skewed linked list**. Every operation becomes O(n).

This is the fundamental weakness of naive BSTs -- and the motivation for **balanced BSTs** (AVL trees, Red-Black trees), which we will study in the next lectures.

Part 4: BST Deletion -- The Three Cases

Why is Deletion Harder Than Insertion?

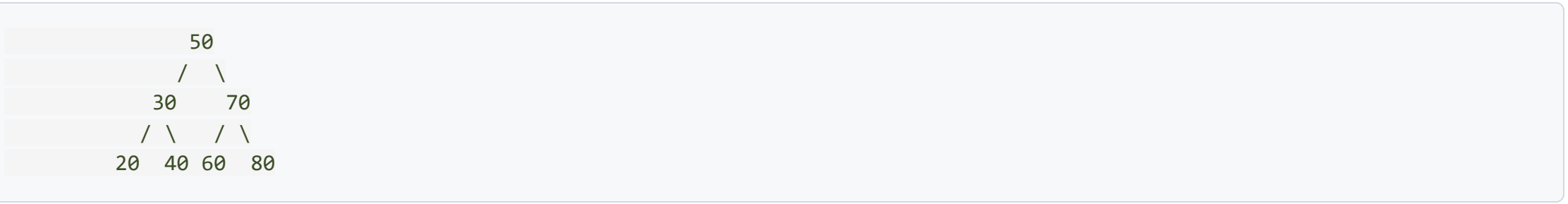
Insertion is easy because we always add a **leaf** -- nothing else in the tree changes.

Deletion is harder because removing a node might **break the tree** -- it might disconnect children from the rest of the tree, or violate the BST property.

We must handle **three cases** based on how many children the node has:

Case	Node to Delete Has	Difficulty
1	**No children** (leaf)	Simple
2	**One child**	Moderate
3	**Two children**	Tricky

Let us examine each case on our reference BST:



Case 1: Deleting a Leaf Node

Delete 20 (a leaf -- no children):

Simply remove the node and set the parent's pointer to NULL.

```
Before:                          After:
       50                                50
      /  \                              /  \
    30    70                         30    70
   / \   / \                         \   / \
 [20] 40 60  80                      40 60  80

Just remove 20 and set 30->left = NULL
```

This is the easiest case. No restructuring needed. The BST property is preserved because removing a leaf does not affect any other node's position.

Case 2: Deleting a Node with One Child

Delete 30 (has two children in our tree -- let us modify the example).

Consider deleting **70** from this tree where 70 has only a right child:

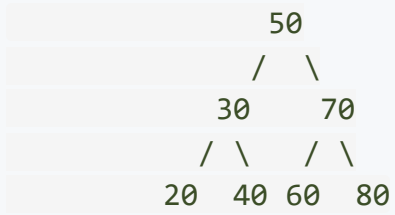
```
      50                    Delete 70:           50
     /  \                                      /  \
   30    70                 Replace 70         30    80
  / \      \                with its only
 20  40     80              child (80)
```

The rule: Replace the deleted node with its **only child**. The child "gets promoted" to take the deleted node's place.

Why does this work? If node X has only a right child R, then everything in R's subtree is greater than X's parent (if X was a right child) or less than X's parent (if X was a left child). The BST property is preserved because R was already in the correct position relative to X's parent.

Case 3: Deleting a Node with Two Children

Delete 50 (the root -- has both left and right children):



We cannot simply remove 50 -- both subtrees would become disconnected. We need a **replacement** value that:

1. Keeps everything in the left subtree smaller
2. Keeps everything in the right subtree larger

Two valid choices:

- **In-order successor:** The smallest value in the right subtree (next value in sorted order)
- **In-order predecessor:** The largest value in the left subtree (previous value in sorted order)

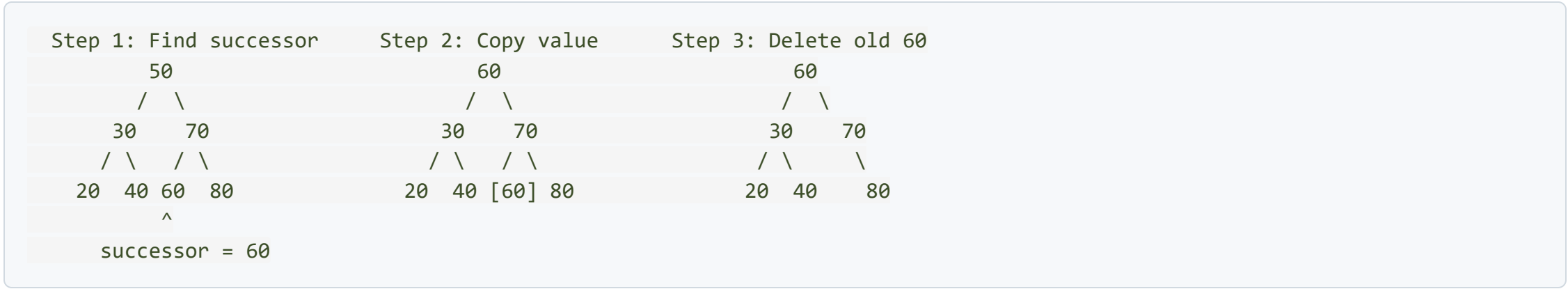
Case 3: Using the In-Order Successor

The **in-order successor** of 50 is the **smallest node in its right subtree** -- which is **60**.

Step 1: Find the in-order successor (60) -- go right once, then go left as far as possible.

Step 2: Copy the successor's value (60) into the node being deleted (50).

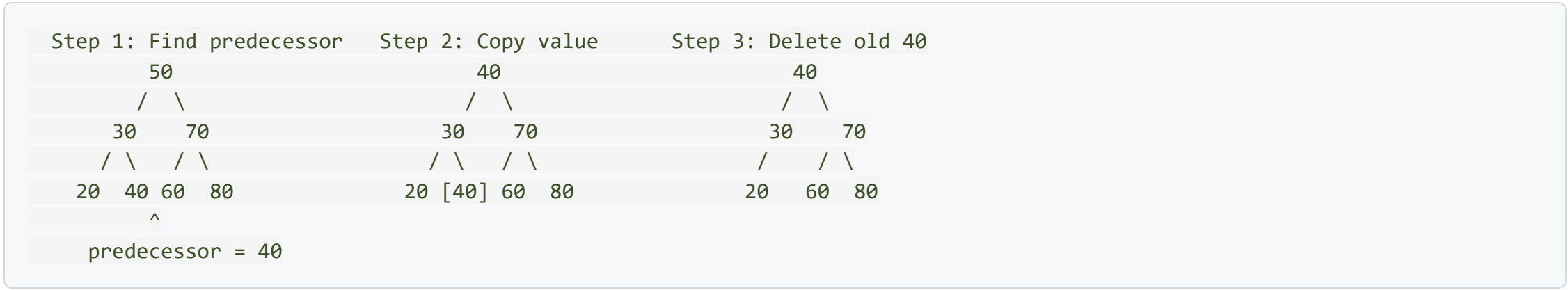
Step 3: Delete the successor (60) from its original position -- this is now a Case 1 or Case 2 deletion (the successor has at most one child).



Final result: The BST property is preserved because 60 was the next value after 50 in sorted order.

Case 3: Using the In-Order Predecessor

Alternatively, we could use the **in-order predecessor** of 50, which is the **largest node in the left subtree** -- that is **40**.



Both approaches are valid. The choice is a convention -- most textbooks use the in-order successor, but either works.

Deletion: Code

```c
// Find the node with the minimum value in a subtree
struct Node* findMin(struct Node* node) {
    while (node->left != NULL) {
        node = node->left;
    }
    return node;
}

struct Node* deleteNode(struct Node* root, int key) {
    if (root == NULL) return NULL;

    if (key < root->data) {
        root->left = deleteNode(root->left, key);
    } else if (key > root->data) {
        root->right = deleteNode(root->right, key);
    } else {
        // FOUND the node to delete
        if (root->left == NULL) {           // Case 1 & 2: no left child
            struct Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {    // Case 2: no right child
            struct Node* temp = root->left;
            free(root);
            return temp;
        }
        // Case 3: two children
        struct Node* succ = findMin(root->right);  // In-order successor
        root->data = succ->data;                   // Copy successor's value
        root->right = deleteNode(root->right, succ->data); // Delete successor
    }
    return root;
}
```

Deletion Code: How the Cases Map

```
// Case 1 (leaf) and Case 2 (one child) are handled together:
if (root->left == NULL) {            // No left child
    struct Node* temp = root->right; // Could be NULL (case 1) or a node (case 2)
    free(root);
    return temp;                     // Return the child (or NULL) to parent
}
```

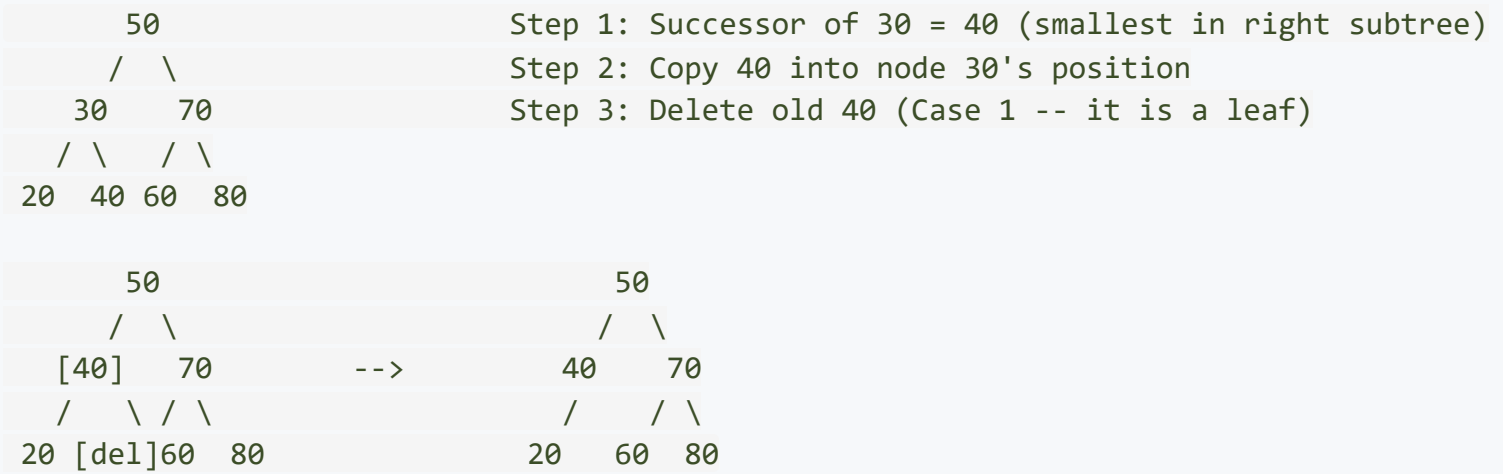
This is elegant: A leaf node has `right == NULL`, so we return NULL to the parent. A node with only a right child returns that right child to the parent. Both cases are handled by the same two lines of code.

```
// Case 3 (two children):
struct Node* succ = findMin(root->right);  // 1. Find smallest in right subtree
root->data = succ->data;                   // 2. Overwrite with successor's value
root->right = deleteNode(root->right, succ->data); // 3. Delete the successor
```

Step 3 is recursive, but the successor has at most one child (it cannot have a left child, because then *that* child would be smaller). So this recursive call will hit Case 1 or Case 2 -- never Case 3 again.

Deletion: Complete Worked Example

Delete **30** from the tree (Case 3 -- two children):



Result: The BST property is intact -- 20 < 40 < 50 < 60 < 70 < 80.

Part 5: Complexity and The Balance Problem

BST Operation Complexities

Every BST operation follows a path from the root toward a leaf. The performance depends entirely on the **height** of the tree.

Operation	Best Case (balanced)	Worst Case (skewed)
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)
Find Min/Max	O(log n)	O(n)
In-order Traversal	O(n)	O(n)

Space: O(n) for the tree itself, O(h) extra space for recursive operations.

The critical insight: For a BST with n nodes:

- **Best** height: $\lfloor \log_2 n \rfloor$ (balanced, complete tree)
- **Worst** height: $n - 1$ (degenerate, skewed tree)

The Shape Problem: Same Data, Different Trees

The **same 7 values** can produce drastically different BSTs depending on insertion order:

```
Insert order: 50, 30, 70, 20, 40, 60, 80        Insert order: 20, 30, 40, 50, 60, 70, 80

           50                                          20
          /  \                                           \
        30    70          Height = 2                     30
       / \   / \          Search = O(log n)                \
     20  40 60  80                                         40
                                                             \
                                                              50    Height = 6
                                                                \   Search = O(n)
                                                                60
                                                                  \
                                                                  70
                                                                    \
                                                                    80
```

Same data. Same structure definition. Wildly different performance. This is the fundamental weakness of unbalanced BSTs.

Can We Prevent Degeneration?

The question: Is it possible to keep the tree balanced automatically after every insert and delete?

The answer: Yes! This is exactly what **self-balancing BSTs** do:

Balanced BST	Year	Idea
AVL Tree	1962	Keep $
Red-Black Tree	1972	Color nodes red/black to enforce balance rules
Splay Tree	1985	Move recently accessed nodes to the root

All of these guarantee O(log n) height by performing **rotations** -- local restructuring operations that change the tree's shape without violating the BST property.

Next lecture: We will study AVL trees in detail -- the first and most intuitive self-balancing BST.

BST vs Other Data Structures

Structure	Search	Insert	Delete	Sorted?
Unsorted Array	O(n)	O(1)	O(n)	No
Sorted Array	O(log n)	O(n)	O(n)	Yes
Linked List	O(n)	O(1)	O(n)	Depends
BST (balanced)	**O(log n)**	**O(log n)**	**O(log n)**	**Yes**
BST (worst)	O(n)	O(n)	O(n)	Yes

The BST (when balanced) is the first structure we have seen that achieves O(log n) for all three major operations -- search, insert, and delete -- while also keeping data in sorted order.

This is why BSTs are the foundation of many important data structures: sets, maps, dictionaries, priority queues, and database indexes.

Key Takeaways

What We Covered Today

1. The BST Property

- Left subtree values < node < right subtree values
- In-order traversal always gives sorted output

2. BST Search

- Binary search on a tree structure -- go left or right at each step
- O(h) comparisons -- O(log n) when balanced, O(n) when skewed

3. BST Insertion

- New nodes always become leaves -- search for the right spot, then attach
- Insertion order determines tree shape

4. BST Deletion -- Three Cases

- Leaf: just remove
- One child: promote the child
- Two children: replace with in-order successor (or predecessor), then delete the successor

5. The Balance Problem

- Sorted input creates a degenerate tree (linked list)
- Self-balancing BSTs (AVL, Red-Black) solve this -- coming next!

Review Questions

1. Insert the values **15, 10, 20, 8, 12, 17, 25** into an empty BST (in the given order). Draw the resulting tree.
2. What is the in-order traversal of the tree you built in question 1? Why is this significant?
3. Delete the value **15** from the tree in question 1 using the in-order successor method. Show each step.
4. Insert the values **1, 2, 3, 4, 5, 6, 7** into an empty BST. What does the resulting tree look like? What is its height?
5. Given the same 7 values from question 4, what insertion order would produce a **balanced** BST of minimum height?
6. A BST has 1000 nodes and is perfectly balanced. What is the maximum number of comparisons needed to find any value?
7. Explain why the in-order successor of a node with two children can never have a left child.

Next Lecture

Balanced BSTs and AVL Tree Rotations

- The balance factor and when a tree becomes "too lopsided"
- Four rotation cases: LL, RR, LR, RL
- Automatic rebalancing after every insertion
- Guaranteeing O(log n) performance no matter what